Project Title

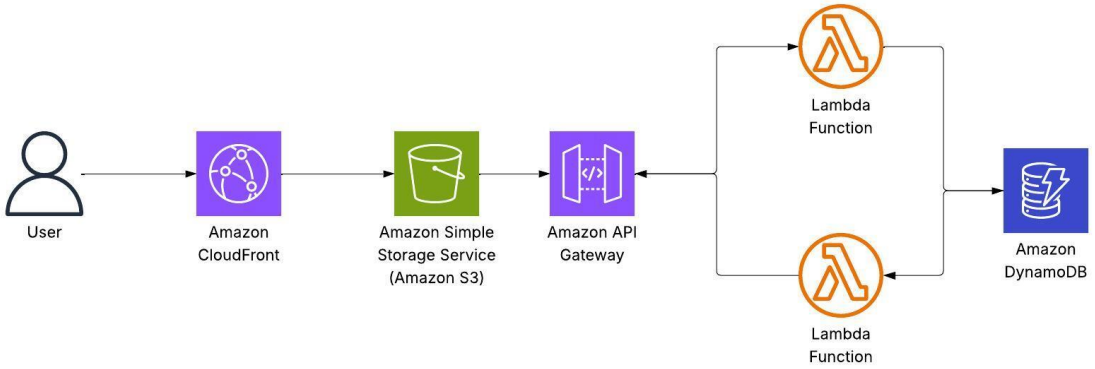
- AWS Serverless Contact Form Submission System
- A scalable, secure, and serverless contact form built on AWS

Project Goal

Objective:

- - Collects student data
- - Stores it securely
- - Displays submitted data dynamically
- - Uses 100% AWS serverless infrastructure

Architecture Diagram



1.Amazon CloudFront

- Delivers your frontend globally with **HTTPS, low latency**, and edge caching.

Attached to S3 bucket

2. Amazon S3 (Simple Storage Service)

- Hosts the frontend files (index.html, script.js) as a static website.

3.Amazon API Gateway

- Acts as the **REST API** for frontend to talk to backend securely.
- Handles POST and GET requests
- Integrated with Lambda

4.AWS Lambda

 Runs backend logic **without servers**.

Written in Python

POST: Saves form data to DynamoDB

GET: Retrieves all data from DynamoDB

5.Amazon DynamoDB

- Stores student data in a **NoSQL table**
- Secure and serverless

6.IAM (Identity and Access Management)

- Controls which services can talk to each other securely.
- Lambda granted access to DynamoDB

What is the Use of This Architecture?

1. Fully Serverless

- No servers to manage, patch, or scale manually.
- AWS automatically handles infrastructure, scaling, availability, and redundancy.

2. Highly Scalable

- Each component auto-scales independently — suitable for millions of users.

3. Secure by Design

- Uses IAM, OAC, CORS, and HTTPS
- Only CloudFront can access S3 (OAC)
- IAM roles restrict access to only required actions
- CORS headers limit frontend access to backend securely
- HTTPS ensures encrypted communication

4. Cost Efficient (Pay-as-you-go)

- You only pay for what you use.
- No idle server costs
- DynamoDB and Lambda charge per request
- S3 + CloudFront charge based on traffic and bandwidth

Each AWS service handles one responsibility:

- **S3** → Static hosting
- **API Gateway** → Routing requests
- **Lambda** → Executing logic
- **DynamoDB** → Storing data
- This modular approach makes it **easier to debug, scale, or upgrade**.

Final Summary:

- "This architecture provides a **scalable, secure, cost-efficient, and highly available** solution — perfect for real-time web apps that need to collect and manage user data reliably.

1. Create DynamoDB Table

- Table name: studentData
- Partition key: studentid (String)



2. Create 2 Lambda Function (Python)

- One for get the data from dynamodb one for to post the data for dynamodb
- Runtime: Python 3.12+
- Permissions: created IAM role for lambda to assess dynamodb



- Lambda Code to put the data in dynamodb

```
import json
import boto3

# Create a DynamoDB object using the AWS SDK
dynamodb = boto3.resource('dynamodb')
# Use the DynamoDB object to select our table
table = dynamodb.Table('studentData')

# Define the handler function that the Lambda service will use as an entry point
def lambda_handler(event, context):
    # Extract values from the event object we got from the Lambda service and store in variables
    student_id = event['studentid']
    name = event['name']
    student_class = event['class']
    age = event['age']

    # Write student data to the DynamoDB table and save the response in a variable
    response = table.put_item(
        Item={
            'studentid': student_id,
            'name': name,
            'class': student_class,
            'age': age
        }
    )

    # Return a properly formatted JSON object
    return {
        'statusCode': 200,
        'body': json.dumps('Student data saved successfully!')
    }
```

- Lambda Code to get the data from dynamodb

```
import json
import boto3

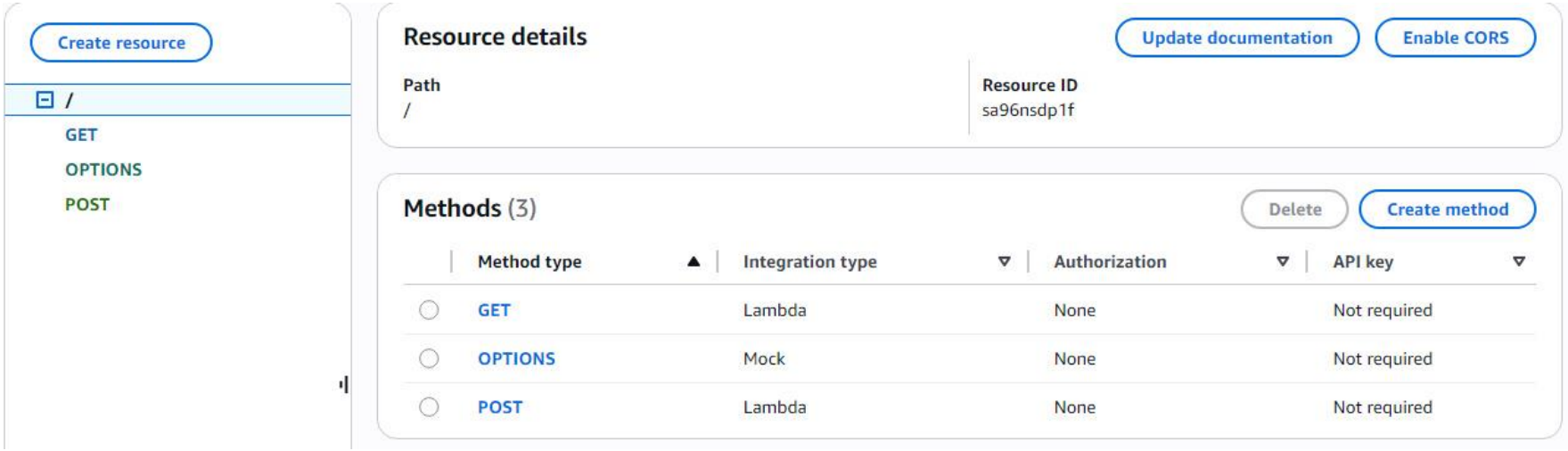
def lambda_handler(event, context):
    dynamodb = boto3.resource('dynamodb')
    table = dynamodb.Table('studentData')  # change to your actual table name

    try:
        response = table.scan()
        data = response.get('Items', [])

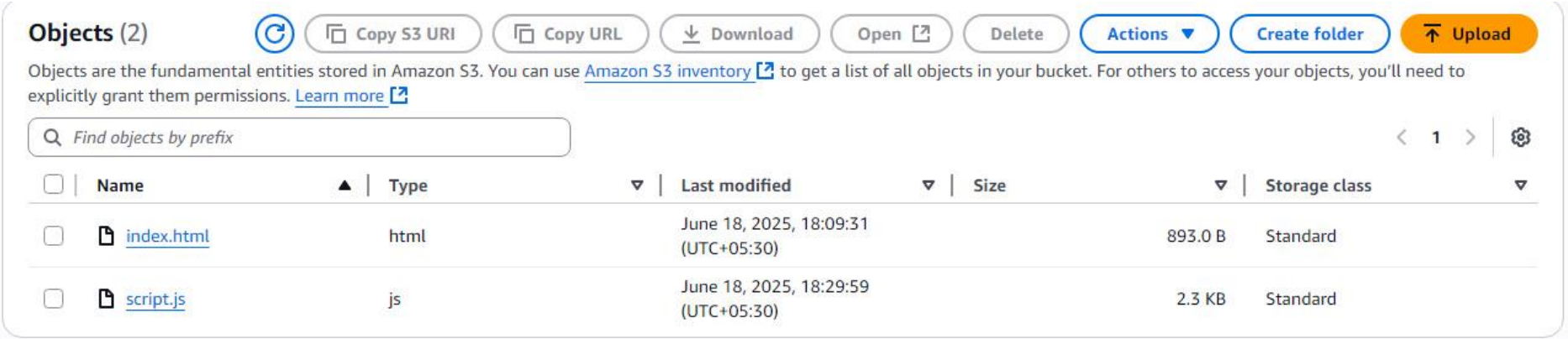
        return {
            "statusCode": 200,
            "headers": {
                "Access-Control-Allow-Origin": "*"
            },
            "body": json.dumps(data)
        }

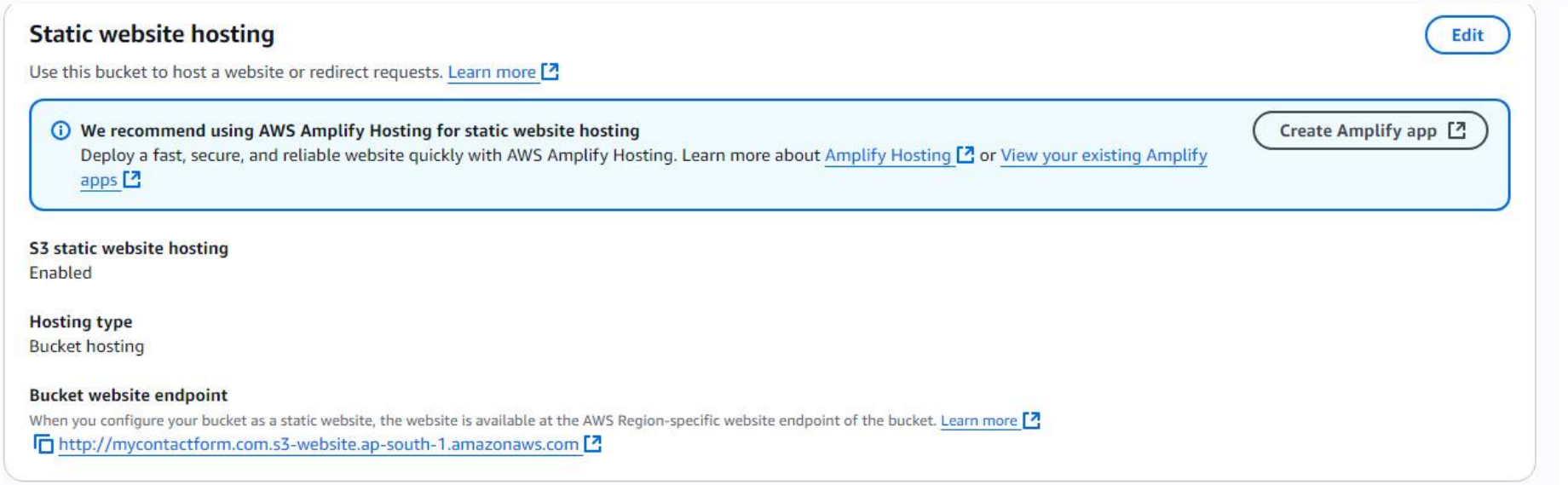
    except Exception as e:
        return {
            "statusCode": 500,
            "headers": {
                "Access-Control-Allow-Origin": "*"
            },
            "body": json.dumps({"error": str(e)})
        }
```

- **Create API Gateway**
- REST API → Resource /
- Methods: POST and GET
- Integration: Lambda Proxy
- Enable **CORS** on both methods
- Deploy to stage: prod

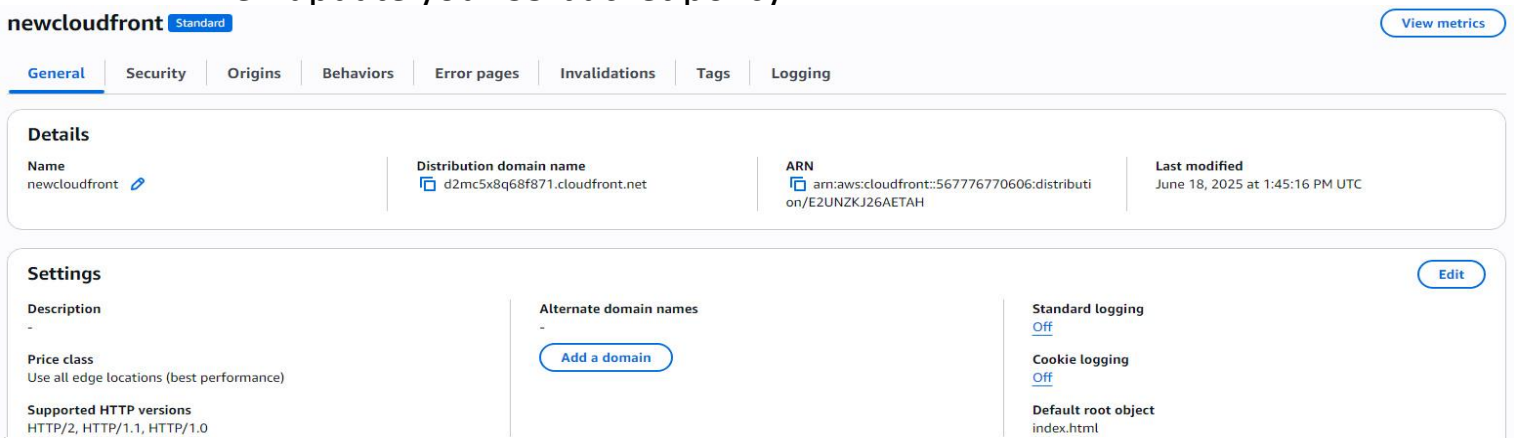


- **Create Static Website on S3**
- Enable Static Website Hosting
- Upload:
  - index.html
  - script.js





- **Create CloudFront Distribution**
- Origin: your S3 bucket
- Enable: Redirect HTTP to HTTPS
- Set default root object: index.html
- Use **Origin Access Control (OAC)**:
  - Then update your S3 bucket policy:




```
{
  "Version": "2008-10-17",
  "Id": "PolicyForCloudFrontPrivateContent",
  "Statement": [
    {
      "Sid": "AllowCloudFrontServicePrincipal",
      "Effect": "Allow",
      "Principal": {
        "Service": "cloudfront.amazonaws.com"
      },
      "Action": "s3:GetObject",
      "Resource": "arn:aws:s3:::mycontactform.com/*",
      "Condition": {
        "StringEquals": {
          "AWS:SourceArn": "arn:aws:cloudfront::567776770606:distribution/E2UNZKJ26AETAH"
        }
      }
    }
  ]
}
```

Final output :

Student Contact Form

Studentid:
7
Name:
john
Class:
12
Age:
17

Save Student | Get Students

Student data saved successfully!

Studentid	Name	Class	Age

Student Contact Form

Studentid:
7
Name:
john
Class:
12
Age:
17

Save Student | Get Students

Student data saved successfully!

Studentid	Name	Class	Age
22	jayanth	12	17
2	shiva	10	15
1	uppi	12	17
35	subhash	15	20
5	nan	5	8
7	john	12	17

Backend data saved inside dynamodb

Completed · Items returned: **6** · Items scanned: **6** · Efficiency: **100%** · RCUs consumed: **2**

Table: studentData - Items returned (6)

Scan started on June 18, 2025, 19:27:49

studentid *(String)*	age	class	name
22	17	12	jayanth
2	15	10	shiva
1	17	12	uppi
35	20	15	subhash
5	8	5	nan
7	17	12	john

PROBLEMS FACED

- **POST not working, GET shows undefined in table**
- **Problem:**
  Table displayed “undefined” instead of data after clicking “Get Students”.
  unable to save data
  { every thing was working fine in backed but frontend was having issue}
- **Why it happened:**
  Key mismatch – frontend was using data['studentId'], but data had studentid.
  problems Found in script.js
  Using POST, but your API expects PUT
  API Gateway method is / - PUT
  But in code, you are using:Type: 'POST'
- **How I solved it:**
- Checked key names using console.log(students)
- Fixed JavaScript to match exact keys:
- Created ‘POST’ API method